The Oracle Forge

Facilitator & Team Manual

Weeks 8 – 9 · Arc 4 · Context Engineering & Evaluation Science · April 2026

How to Use This Manual

This manual has two audiences. Facilitators use it to run the two-week sprint: check team health, confirm AI-DLC gate approvals are happening, and apply the evaluation rubric at the end. Team members use it as a working reference during the sprint: infrastructure setup, role playbooks, benchmark submission instructions, and the DAB dataset guide.

Section	Primary audience
Part 1 — Sprint structure and rituals	Everyone
Part 2 — Infrastructure: tenai-infra, server, MCP Toolbox, sandbox	Drivers, Intelligence Officers
Part 3 — Role playbooks: AI-DLC, Karpathy KB, Signal Corps content	All roles
Part 4 — DataAgentBench guide: datasets, setup, submission	Drivers, Intelligence Officers
Part 5 — Evaluation reference: full rubric, checklist	Facilitators, all roles

Part 1 — Sprint Structure and Rituals

The two weeks as a single sprint

There is one sprint, not two. The two weeks have different primary emphases, but there is no separate Week 8 and Week 9 submission. The product is built continuously and submitted at the end of Week 9.

Period	Primary emphasis	State at end of period
Week 8, Days 1–2	Infrastructure and architecture	tenai-infra running, DAB databases loaded, MCP Toolbox configured, Knowledge Base v1 committed, AI-DLC Inception document approved by full team
Week 8, Days 3–5	Core agent build	Agent running on team server, handles at least two DAB database types, basic

		NL-to-query working, first Signal Corps post live
Week 9, Days 1–2	Context engineering depth	All three context layers implemented, Knowledge Base v2 and v3 committed, evaluation harness producing scores against held-out set
Week 9, Days 3–4	Adversarial testing and improvement	Adversarial probe library complete, score improving as measured by harness, regression suite running clean
Week 9, Day 5	Submission	DAB results JSON submitted via GitHub PR, all deliverables packaged, engagement summary compiled, facilitator demo completed

The daily mob session

The mob session is the team's primary working unit. Every working day, the full team works together for at least one hour. One Driver holds the keyboard and AI interface. Everyone else is an active co-pilot: asking questions, answering AI-generated questionnaires, reviewing outputs, and making decisions collectively.

Mob programming is not pair programming with more observers. Everyone is engaged. The driver executes only what the team decides. If the co-pilots cannot follow what the driver is doing, the driver slows down, not the co-pilots. The pace is set by understanding, not by keystroke speed.

Mob session segment	Duration	What happens
Intelligence Officers update	10 min	What is new in the Knowledge Base. What the global ecosystem produced overnight that the team should know. Any new entries in the corrections log.
Signal Corps update	10 min	What went out, what response it got, any community intelligence that changes the technical approach. Resource acquisitions.
Driver-led construction or operations	40 min	Driver holds keyboard and AI tool interface. Team co-pilots: questions aloud, AI questionnaire answers decided collectively, outputs reviewed before committing. AI-DLC gate approvals happen here when due.

Gate approvals: The AI-DLC transitions between Inception → Construction → Operations happen at mob sessions, not in Slack. The team reads the Inception document together, asks hardest questions, approves or sends back. No solo gate crossing.

Part 2 — Infrastructure Setup

The tenai-infra system

Your team's shared infrastructure runs on the tenai-infra system. This is the same infrastructure used in Week 4 (Brownfield Cartographer) — you are not learning new tooling, you are applying familiar tooling at scale across a two-week sprint with a team.

Tenai infrastructure code provides: Tailscale mesh networking so every team member connects to the shared server from any device (laptop, mobile, tablet) without VPN configuration; Gemini CLI conductor for managing parallel AI agent sessions that all team members can observe and direct; parallel git worktrees for running multiple experiments simultaneously without interference; and tmux monitoring for persistent session management across disconnections.

> **Setup instruction:** Follow the tenai-infra installation README exactly. The README is the authoritative source. Do not deviate from it. If the README has a gap or error, Signal Corps should raise it as an issue in the tenai-infra repository — that is a legitimate contribution.

```
# Step 1: Clone tenai-infra on the team server
git clone https://github.com/yabebalFantaye/tenai /shared/tenai-infra
cd /shared/tenai-infra

# Step 2: Follow the installation README
cat README.md   # Read this first — completely

# Step 3: Verify Tailscale mesh
# Each team member installs Tailscale on their device
# and joins the team network using the auth key from the README
tailscale status   # Should show all team devices

# Step 4: Start tmux monitoring session
tmux new-session -s oracle-forge
# All team members can attach: tmux attach -t oracle-forge
```

Loading the DataAgentBench datasets

DataAgentBench provides 12 datasets across 4 database systems. You do not need to load all 12 from day one. Load in this order: first the PostgreSQL databases (most of DAB's queries), then SQLite, then MongoDB, then DuckDB. The benchmark repository provides setup scripts for each.

```
# Clone the DataAgentBench repository
git clone https://github.com/ucbepic/DataAgentBench.git
cd DataAgentBench

# Read the setup guide
cat README.md
```

```
# Install dependencies
pip install -r requirements.txt

# Load PostgreSQL databases (do this first)
bash setup/load_postgres.sh

# Verify: run the example query against the Yelp dataset
python eval/run_query.py --dataset yelp --query 0

# Expected: structured result JSON with query trace
```

The Yelp dataset (included in DAB) is the recommended starting point — it contains multi-source data, nested JSON, missing values, and entity resolution challenges that mirror the full DAB problem space in a contained form. Use it to validate your agent architecture before extending to other datasets.

Google MCP Toolbox configuration

MCP Toolbox for Databases provides the standard interface between your agent and the DAB databases. A single tools.yaml file defines all database connections and tools. Your agent calls tools from this file via the MCP protocol rather than writing raw database drivers per database type.

```
# Download toolbox binary (check googleapis/genai-toolbox for latest version)
export VERSION=0.30.0
curl -O https://storage.googleapis.com/genai-toolbox/v$VERSION/linux/amd64/toolbox
chmod +x toolbox

# Create tools.yaml (template in team repository at mcp/tools.yaml)
./toolbox --config mcp/tools.yaml
# Toolbox runs on http://localhost:5000

# Verify all databases accessible
curl http://localhost:5000/v1/tools | python3 -m json.tool | grep name
```

Your tools.yaml must define connections to all four database types DAB uses. A starter template with the required source and tool definitions for PostgreSQL, SQLite, MongoDB, and DuckDB is in the team repository at mcp/tools.yaml. Drivers add tools as the agent needs them; Intelligence Officers maintain the tools documentation in the Knowledge Base.

Code generation sandbox

Your agent needs a sandbox for executing data transformation code outside the LLM context — the same pattern used in the Week 4 Brownfield Cartographer. The sandbox receives a code plan from the LLM, executes it against the databases, runs validation, and returns structured results. Two options:

Option A — Local container (tenai-infra default)

```
# The tenai-infra system includes a default sandbox container
# See tenai-infra/sandbox/README.md for configuration

# Start sandbox server
python3 sandbox/sandbox_server.py --port 8080

# The agent sends code to POST /execute
# Returns: {result, trace, validation_status, error_if_any}
```

Option B — Cloudflare Workers (free tier)

```
# Install Wrangler CLI
npm install -g wrangler

# Log in (free Cloudflare account)
wrangler login

# Deploy the sandbox worker
cd workers && wrangler deploy
# URL: https://sandbox.[team-name].workers.dev

# Set in team .env
SANDBOX_URL=https://sandbox.[team-name].workers.dev
```

> **Signal Corps task:** Apply for Cloudflare Workers free tier on Day 1. Also check whether Cloudflare has any developer programme credits available. Report to the team at the Day 1 mob session with the outcome and access instructions.

Part 3 — Role Playbooks

Drivers — AWS AI-DLC framework

AI-DLC (AI-Driven Development Life Cycle) is AWS's framework for structured human-AI collaboration. It organises development into three phases with mandatory human approval gates between them. For this challenge, it is the governance layer that keeps the team honest about what AI has actually produced versus what it appears to have produced.

AI-DLC phase	What the team does	Gate to next phase
Inception	Define what will be built: write the Inception document (press-release paragraph, honest FAQ, verifiable definition of done). Answer all AI-generated questions as a full team at the mob session. No individual decisions.	Team reads Inception document together at mob session. Hard questions asked. Explicit group approval given before any code is written. Record approval in mob session log.
Construction	Driver holds keyboard and AI interface. Team co-pilots: directing queries, answering	Driver demonstrates that every item on the definition-of-done checklist is

	questionnaires, reviewing AI outputs before committing. Code is written with the team watching and directing.	verifiably true — not "I think it works" but "here is the evidence it works." Team approves at mob session.
Operations	Document what was built, what changed from the plan, what the harness score is, what the next sprint's Inception should address. Record everything — if the codebase were deleted, this record survives.	Facilitator or team lead reviews operations document. Sprint is complete. Next sprint's Inception begins.

AI-DLC Inception document structure

- **Written as if the sprint is complete. Names specifically what was built, what it can do, and why it matters. Present tense. Hard to write well — that difficulty is the point.**Press release (one paragraph):
- **Questions a user would ask about the product. Honest answers, including what it does not do.**Honest FAQ — user (three Q&A):
- **What could go wrong. What the hardest part is. What dependencies exist. Honest answers.**Honest FAQ — technical (three Q&A):
- **Each decision with the chosen option and a one-sentence reason. No decision by default.**Key decisions (two to three):
- **Numbered, specific, verifiable. Not "agent works" — "agent returns the correct answer to query X in under 10 seconds" with the specific query named.**Definition of done (five to eight items):

Intelligence Officers — Karpathy Knowledge Base method

The LLM Knowledge Base is the team's most important persistent asset. It is a set of structured documents that can be injected directly into an LLM context window, giving it working knowledge of the domain without explanation or preamble. Andrej Karpathy's method for building LLM knowledge bases: minimum content, maximum precision, verified by injection test.

The discipline is removal, not accumulation. Every document in the KB should be tested: inject it into a fresh LLM context and ask a question it should answer. If the document does not produce the correct answer, revise it or remove it. A KB that grows without being tested becomes noise that degrades the agent.

Knowledge Base structure for this challenge

- **Claude Code three-layer memory system (MEMORY.md as index, topic files on demand, session transcripts searchable). autoDream consolidation pattern. Tool scoping philosophy. OpenAI data agent six-layer context and table enrichment design. Each document: maximum 400 words, written as "here is how this works," verified by injection test.**kb/architecture/:
- **DAB schema descriptions per dataset. Known query patterns that work across database types. Ill-formatted join key glossary (how customer IDs appear differently across PostgreSQL and MongoDB in each dataset). Unstructured field inventory (which fields contain free text requiring transformation). Domain term definitions.**kb/domain/:

- **DAB query format, scoring method (pass@1), submission requirements. Evaluation harness schema. The four DAB failure categories and which probe types test each.**kb/evaluation/:
- **Running structured log: [query that failed] → [what was wrong] → [correct approach]. Written by Drivers after every observed agent failure. Read by the agent at session start. This is the self-learning loop — the mechanism by which the agent improves from its own errors without retraining.**kb/corrections/:

Injection test protocol

```
# Test every KB document before committing

# 1. Take the document text
# 2. Start a fresh LLM session with ONLY that document as context
# 3. Ask a question the document should answer
# 4. Grade: correct answer = document passes. Wrong = revise or remove.

# Example test for kb/domain/join_keys.md:
# Question: "How is CustomerID formatted in the Yelp PostgreSQL database
#            versus the MongoDB reviews collection?"
# Expected: specific format strings from your document
# If the LLM cannot answer from the document alone, the document fails.
```

KB quality criteria

Criterion	Test for compliance
Injected, not summarised	Can you paste the document directly into a context window and get correct answers without additional explanation?
Specific to this problem	Have you removed everything the LLM already knows from pretraining? Only include what is specific to DAB, these databases, this agent.
Maintained, not archived	Does the KB have a CHANGELOG.md? Are outdated entries removed when the database or agent changes? Growth without removal = noise.
Verified before committing	Has every document passed an injection test? Documents that fail injection tests should be revised before merging.

Signal Corps — content calendar and community strategy

Signal Corps is an amplification and intelligence role, not a documentation role. The goal is to participate in the global conversation that is happening right now about exactly the problem your team is working on — data agents, multi-database retrieval, the Claude Code architecture — and to do so with enough technical credibility that the conversation responds.

Technical credibility requires attendance at mob sessions and genuine understanding of the team's technical state. A Signal Corps member who has not read the DAB paper cannot write a credible thread about it. A Signal Corps member who does not attend mob sessions cannot accurately represent what the team found.

Week 8 content plan

Output	Description and guidance
Day 1 — Resource audit	Apply for Cloudflare Workers free tier. Identify 5 X accounts posting about data agents, Claude Code, or BIRD/DAB benchmarks. Subscribe to DAB repository. Note which subreddits and Discord servers are active.
Day 2 — First X thread	Comment on a notable post about Claude Code architecture or data agents. Contribute a specific technical observation from the team's study of the KB. Name the specific thing you learned, not a generic opinion.
Day 3 — Internal Slack daily + community entry	Daily Slack post in team channel. One substantive comment on Reddit (r/MachineLearning or r/LocalLLaMA) or Discord (Hugging Face, DAB community if accessible). Save link in community log.
Day 4 — Second X thread	Post about what your team is building: the DAB benchmark, the multi-database architecture decision, something unexpected you hit in Day 3. In-progress engineering posts consistently outperform finished-product announcements in technical communities.
Day 5 — Weekly engagement summary	Compile all post links, notable responses, resources obtained. Present at mob session. Report any community intelligence that changes the team's technical direction.

Week 9 content plan

Output	Description and guidance
Day 1 — Article draft	Each Signal Corps member drafts a personal technical post (600–1000 words). Topic: one specific thing learned, one failure understood, one architectural decision made. Not a project summary. Examples: "What DAB taught me about enterprise data reality" or "The self-correcting execution loop: how we taught an agent to debug its own queries."
Day 2 — Benchmark X thread	Post about the team's benchmark submission process — the setup, the first scores, what the evaluation harness measures. The DAB benchmark community is small and active. Tag the DAB repository if appropriate.
Day 3 — Article published	Publish on LinkedIn or Medium. Share link on X with a thread summarising the single most important point. Signal Corps member two posts their article if ready; if not, schedule for Day 4.
Day 4 — Final community thread	Post benchmark results once submitted. Reference the DAB leaderboard. Engage with any responses within the two-week window.
Day 5 — External engagement summary	Complete the engagement portfolio: all posts with links, any reach metrics, notable responses, list of any community intelligence that changed the team's technical approach. This is a deliverable — not a post-mortem.

What makes a credible technical post

- **Name the query, the database, the failure mode, the fix. "We found that MongoDB aggregation pipelines require explicit field projection before joining with PostgreSQL results" is credible. "We improved our multi-database handling" is not.**Specific, not general:
- **Describing a failure you diagnosed and fixed is more credible than describing only successes. The technical community reads for learning, not marketing.**Honest, not promotional:
- **Reply to responses. Quote other practitioners. Ask a follow-up. Posts that feel like conversation get more traction than posts that feel like announcements.**Engaged, not broadcast:
- **Reference the papers, the leaderboards, the code, the PR. Links signal that you are operating in the actual technical community.**Linked to evidence:

Part 4 — DataAgentBench Guide

Understanding what DAB actually tests

DAB is the first benchmark designed to test AI agents on realistic enterprise data workloads. The paper is required reading for Intelligence Officers before the first mob session. The specific things that make DAB different from previous benchmarks are also what make your agent hard to build — so understanding them before you design the architecture saves significant rework.

DAB requirement	What it means in practice	Which previous project it connects to
Multi-database integration	A single query may require fetching from PostgreSQL and MongoDB, then reconciling the results. Your agent must route sub-queries to the correct database type, translate between query dialects, and merge results without data loss.	Week 4 Brownfield Cartographer: multi-agent routing across tool types. The Conductor/worker pattern applies here at the database level.
Ill-formatted join keys	Customer IDs in PostgreSQL may be integers. The same customers in MongoDB may be stored as "CUST-00123" strings. Your agent must detect the format mismatch and resolve it before attempting a join — without being told explicitly.	Week 6 Data Contract Enforcer: schema enforcement and format validation. The format detection logic from the Enforcer applies directly.
Unstructured text transformation	Some DAB queries require extracting structured facts from free-text fields — support notes, review text, product descriptions. The agent must perform extraction before the fact can be used in a calculation.	Week 3 Document Intelligence Refinery: structured extraction from unstructured sources. This is the same pipeline, applied to database field values rather than uploaded documents.
Domain knowledge	Answering correctly requires knowing things not present in the schema — industry terminology, fiscal calendar conventions, status code meanings. The agent must	Week 7 Context Architect: measuring and optimising context quality. The institutional knowledge layer in the KB is the

	have this knowledge before the query arrives.	production application of what Week 7 made measurable.

DAB evaluation setup

```
# The DAB repository contains all datasets and evaluation scripts
git clone https://github.com/ucbepic/DataAgentBench.git
cd DataAgentBench && pip install -r requirements.txt

# Dataset overview
python eval/list_datasets.py
# Output: 12 datasets, their domains, DB types, and query count

# Run your agent on a single query to test the interface
# Your agent must accept: {question, available_databases, schema_info}
# Your agent must return: {answer, query_trace, confidence}

# Run the full evaluation (54 queries, 50 trials each)
# This takes time — plan for at least 2 hours
python eval/run_benchmark.py \
  --agent your_agent_module \
  --trials 5 \
  --output results/your_team_results.json

# Check your pass@1 score
python eval/score.py --results results/your_team_results.json
```

Benchmark submission

Submission is a GitHub pull request to the DataAgentBench repository. The PR contains your results JSON file and a brief description of your agent architecture. This is a public contribution — other researchers will be able to see what your agent achieved and how it was built.

```
# Prepare submission
cp results/your_team_results.json submission/team_[name]_results.json

# Create AGENT.md describing your agent (required for PR)
# Include: architecture overview, key design decisions, what worked, what did not

# Fork the repository on GitHub, commit your files
git add submission/team_[name]_results.json AGENT.md
git commit -m "Add [Team Name] DAB evaluation results"
git push origin main

# Open a Pull Request from your fork to ucbepic/DataAgentBench
# Title: "[Team Name] — TRP1 FDE Programme, April 2026"
# Include: pass@1 score, trial count, brief architecture description
```

Signal Corps coordination: The PR submission is a public milestone. Signal Corps should post about it on X when it is opened — linking to the DAB repository and noting the team's score. This is the moment where the team's work is visible to the benchmark community.

Adversarial probe library format

The adversarial probe library is a structured set of queries designed to expose specific failure modes. Each probe targets one of DAB's four hard requirement categories. Minimum 15 probes, minimum 3 categories. The library is stored as probes/probes.md in the team repository.

Failure category	Example probe	What to document
Multi-database routing failure	Query requires customer revenue from PostgreSQL and their support ticket count from MongoDB. Agent either queries only one database or fails the join.	Query text, databases involved, expected join key, how agent failed (wrong database, failed join, wrong result), fix applied, post-fix score on this query
Ill-formatted key mismatch	Customer IDs are integers in the transaction DB and "CUST-[integer]" strings in the CRM. Agent attempts join without format resolution.	Exact format mismatch, how agent detected or failed to detect it, resolution logic that worked
Unstructured text extraction failure	Query requires counting "negative sentiment mentions" in support notes — requires extraction before counting. Agent returns raw text instead of structured count.	Which field, what extraction was needed, what the agent returned, what extraction approach worked
Domain knowledge gap	Query uses term "active customer" — agent uses row existence as a proxy when the correct definition is "purchased in last 90 days." Answer is wrong but looks correct.	The ambiguous term, the naive interpretation, the correct domain definition, where that definition should live in the KB

Part 5 — Evaluation Reference

Full evaluation rubric

Dimension	Below standard	Solid	Advanced
Technical depth (25%)	Agent does not run or fails on more than half of queries without recovery	Agent handles at least two DAB database types, self-corrects on execution failure, produces verifiable output with query trace	Agent handles three or four DAB database types; harness score improves measurably between Week 8 baseline and final submission

Context engineering (20%)	Agent uses only raw schema; no institutional knowledge, no memory	Three context layers implemented and populated; AGENT.md (or equivalent) loaded at session start; corrections log read and written	KB v3 corrections log demonstrably changes agent behaviour on repeated queries; agent resolves cross-database entity mismatches using KB knowledge
Evaluation science (25%)	No harness; benchmark not submitted	Harness produces scores with query trace; regression suite runs clean; DAB benchmark submitted with score recorded	Score improves measurably between first run and submission as evidenced by harness log; adversarial probe library covers 3+ failure categories with fix documentation
Team compounding (15%)	Roles siloed; repo has commits from one or two members only; no cross-role communication evidence	All three roles active with deliverables; repo has commits from all six members; Slack log shows cross-role communication; AI-DLC gate approvals documented	Signal Corps posts reference team's specific technical findings with accuracy; KB changelog shows Intelligence Officer-Driver feedback producing score improvement; mob session log is complete
Signal Corps quality (15%)	Internal Slack only; no external engagement	Minimum posts and threads completed; at least one article published per Signal Corps member; community participation log has substantive entries	External posts receive community engagement; at least one post brought back actionable technical intelligence to the team; DAB PR attracted external attention

Submission checklist

Submit as a single pull request to the programme repository before end of Week 9. Label: [Team Name] — oracle-forge-final.

1. README.md at root: team members and roles, architecture diagram (hand-drawn and photographed is fine), setup instructions a facilitator can follow on a fresh machine, link to live agent on shared server.
2. agent/ directory: AGENT.md context file, MCP tools.yaml, all agent source files, requirements.txt or equivalent.
3. kb/ directory: Four subdirectories (architecture, domain, evaluation, corrections), each with a CHANGELOG.md and evidence of injection tests (the test queries and expected answers used to verify each document).
4. eval/ directory: Evaluation harness source, score log showing progression from first run to final (minimum two data points), held-out test set with expected answers.
5. probes/probes.md: All 15+ adversarial probes in standard format (query, failure category, expected failure, observed failure, fix applied, post-fix score).
6. planning/ directory: AI-DLC Inception documents for all sprints, with mob session approval records (date, who approved, hardest question asked and its answer).

7. utils/ directory: Shared utility library with README describing each module, usage example, and test.
8. signal/ directory: engagement_log.md with all post links and any available metrics, article text files (or links), community participation log with substantive comment links.
9. results/ directory: DAB results JSON, PR link to the DataAgentBench repository, harness score log, screenshot of leaderboard entry if available.

Key references

Resource	URL
DataAgentBench repository	github.com/ucbepic/DataAgentBench
DataAgentBench paper	arxiv.org/html/2603.20576
Google MCP Toolbox for Databases	github.com/googleapis/genai-toolbox
Claude Code architecture analyses	github.com/sanbuphy/claude-code-source-code (docs/en/)
OpenAI data agent writeup	openai.com/index/inside-our-in-house-data-agent
Karpathy LLM Knowledge Bases	academy.dair.ai/blog/llm-knowledge-bases-karpathy
AWS AI-DLC framework	aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/
AWS AI-DLC workflows	github.com/awslabs/aidlc-workflows
Cloudflare Workers free tier	workers.cloudflare.com